

CAHIER DES CHARGES

Site Web Builder & Page Builder (CMS)

Construit avec Nuxt 4 · Vue 3 · TailwindCSS
Un CMS moderne, simple et livrable à vos clients

Version	1.0 — Spécification initiale
Date	2025
Statut	En cours de validation
Destinataire	Équipe de développement / Client

1. Introduction et Contexte

Ce document constitue le cahier des charges technique et fonctionnel d'une solution CMS (Content Management System) sous forme de Site & Page Builder. L'objectif est de permettre à une agence web de déployer rapidement un site web professionnel et personnalisé pour ses clients, tout en leur offrant une interface d'administration simple pour gérer l'ensemble du contenu de manière autonome.

1.1 Problématique

Les clients finaux ont besoin de gérer leur site web sans connaissances techniques. Les solutions existantes (WordPress, Webflow...) sont soit trop complexes, soit trop rigides ou coûteuses. Une solution sur-mesure, légère et moderne basée sur Nuxt 4 permettra de livrer un produit maintenable, évolutif et totalement maîtrisé par l'agence.

1.2 Objectifs Principaux

- Fournir une interface d'administration intuitive (Back-Office) accessible uniquement aux admins

- Permettre la création et la mise en page de pages web via un éditeur visuel (drag & drop de blocs)
- Gérer le contenu (textes, images, liens) directement en ligne sans toucher au code
- Activer ou désactiver des pages, blocs, et fonctionnalités (modules) à la demande
- Gérer les demandes de contact et de devis directement depuis l'administration
- Offrir un outil réutilisable et configurable pour chaque nouveau projet client

1.3 Public Cible

- Utilisateurs principaux : Clients finaux sans compétences techniques (PME, artisans, commerces)
- Utilisateurs techniques : Développeurs/agence pour la mise en place initiale et la configuration

2. Stack Technique

2.1 Architecture Globale

Architecture JAMstack moderne : Nuxt 4 en mode full-stack (SSR + API Routes). Un seul dépôt git, un seul projet, un seul déploiement.

Couche	Technologies	Rôle
Frontend	Nuxt 4, Vue 3, TailwindCSS	Interface client + Admin
Backend/API	Nuxt Server Routes (H3)	API REST interne
Base de données	SQLite (dev) / PostgreSQL (prod)	Persistance des données
ORM	Drizzle ORM	Accès et migration BDD
Auth	nuxt-auth-utils + bcrypt	Gestion des sessions admin
Médias	Cloudinary ou stockage local	Upload images/fichiers
Email	Resend ou Nodemailer	Formulaires de contact/devis
Déploiement	Vercel / VPS / Docker	Mise en production

2.2 Structure du Projet

Le projet suivra la structure recommandée par Nuxt 4 avec une séparation claire entre les couches :

- app/ — Fichiers frontend (pages, composants, layouts, composables)

- server/ — API Routes H3, middlewares, utils serveur
- server/db/ — Schémas Drizzle ORM + migrations
- public/ — Fichiers statiques (favicon, images publiques)
- content/ — Contenus markdown (si module Blog activé)
- shared/ — Types TypeScript partagés front/back

2.3 Choix Techniques Justifiés

Nuxt 4

Framework Vue.js full-stack moderne avec rendu SSR/SSG hybride, optimisé pour le SEO, avec un système de routage automatique et une DX (expérience développeur) excellente. La version 4 apporte de meilleures performances et une architecture plus claire.

Drizzle ORM

ORM TypeScript léger et performant, compatible SQLite (dev local) et PostgreSQL (production). Les migrations sont gérées en code, ce qui simplifie le déploiement et la maintenance à long terme.

TailwindCSS

Framework CSS utilitaire qui permet de construire des interfaces personnalisées rapidement. Le configurateur de thème du CMS s'appuiera sur les variables CSS pour permettre des changements de couleurs et polices sans recompilation.

3. Fonctionnalités et Modules

3.1 Récapitulatif des Modules

Module	Fonctionnalités	Activable
Pages	Créer, éditer, activer/désactiver des pages	Non (socle)
Blocs de contenu	Hero, texte, image, galerie, CTA...	Non (socle)
Navigation	Menu, liens, ordre, visibilité	Non (socle)
Contact	Formulaire + gestion des messages reçus	Oui
Devis	Formulaire de devis + statuts + suivi	Oui
Galerie	Albums photos, gestion uploads	Oui
Blog/Actualités	Articles, catégories, publication	Oui
SEO	Méta-tags, Open Graph, sitemap, robots.txt	Oui (partiel)
Multi-langue	Traductions i18n (optionnel)	Optionnel

3.2 Module Socle — Gestion des Pages

3.2.1 Gestion des Pages

- Créer une nouvelle page avec : titre, slug URL, méta SEO (title, description)
- Modifier l'ordre des pages dans la navigation
- Activer / désactiver une page (la page désactivée ne sera plus accessible sur le site public)
- Dupliquer une page existante comme modèle
- Protéger une page par mot de passe (optionnel)
- Prévisualiser une page avant publication

3.2.2 Éditeur de Blocs (Page Builder)

Le cœur du système. Chaque page est composée d'une liste ordonnée de blocs. L'éditeur permet :

- Ajouter un bloc depuis une bibliothèque de blocs disponibles
- Réordonner les blocs par drag & drop
- Activer / désactiver un bloc sans le supprimer
- Modifier le contenu d'un bloc via un panneau latéral de configuration
- Supprimer un bloc

3.2.3 Bibliothèque de Blocs Disponibles

- Hero — Image de fond, titre principal, sous-titre, bouton CTA
- Texte Simple — Contenu riche (WYSIWYG : gras, italique, liens, listes)
- Image + Texte — Disposition côte à côte (image gauche ou droite)
- Galerie d'images — Grille d'images avec lightbox
- Cards / Services — Grille de cartes avec icône, titre, description
- Témoignages — Carrousel ou grille de témoignages clients
- CTA (Call to Action) — Bloc d'appel à l'action avec bouton
- Vidéo — Embed YouTube/Vimeo ou vidéo hébergée
- Séparateur / Spacer — Espace visuel entre les sections
- Formulaire de Contact — Intégration du module Contact
- Formulaire de Devis — Intégration du module Devis
- Carte / Map — Embed Google Maps avec adresse configurable
- HTML brut — Bloc pour intégrer du code personnalisé (admin uniquement)

3.3 Module Contact

- Formulaire configurable : champs activables/désactivables (nom, email, téléphone, message, objet)
- Envoi d'email de notification à l'adresse configurée dans les paramètres

- Envoi d'un email de confirmation automatique à l'expéditeur
- Tableau de bord des messages reçus dans l'admin
- Statuts des messages : Non lu / Lu / Archivé
- Répondre directement depuis l'interface admin (optionnel)
- Protection anti-spam : honeypot + reCAPTCHA v3 (optionnel)

3.4 Module Devis

- Formulaire de demande de devis configurable (champs personnalisables)
- Possibilité d'ajouter des prestations sous forme de liste à cocher
- Tableau de gestion des devis dans l'administration
- Statuts configurables : Nouveau / En cours / Accepté / Refusé / Archivé
- Export CSV des devis
- Notification email à la réception d'un nouveau devis
- Accusé de réception automatique au client

3.5 Module Galerie

- Création d'albums photos nommés
- Upload multiple d'images (avec optimisation automatique via Sharp ou Cloudinary)
- Réorganisation des images par drag & drop
- Activation / désactivation d'un album
- Affichage public avec lightbox

3.6 Module Blog / Actualités

- Création d'articles avec éditeur de contenu riche
- Catégories et tags
- Image de couverture
- Statut : Brouillon / Publié / Planifié
- URL permalink configurable (slug)
- Méta SEO par article

4. Interface d'Administration (Back-Office)

4.1 Principes d'Interface

Le back-office doit être sobre, intuitif et ne pas nécessiter de formation longue. Il doit être accessible depuis un navigateur sur ordinateur et tablette. L'accent est mis sur la simplicité et la clarté plutôt que sur l'exhaustivité des options.

Principe directeur : « Si le client doit poser une question pour réaliser une action courante, l'interface a échoué. »

4.2 Structure de Navigation

- Tableau de bord — Vue d'ensemble : derniers messages, devis, statistiques rapides
- Pages — Gestion et édition des pages du site
- Navigation — Gestion du menu principal et footer
- Médias — Bibliothèque de fichiers et d'images uploadées
- Contact — Messagerie des formulaires de contact
- Devis — Gestion des demandes de devis
- Blog — Si le module est activé
- Galerie — Si le module est activé
- Apparence — Thème : couleurs, polices, logo, favicon
- Paramètres — Infos du site, email, modules actifs, comptes admin

4.3 Éditeur de Page (Page Builder)

Vue Éditeur

L'interface de l'éditeur de page sera en deux panneaux :

- Panneau gauche (1/3) : Liste des blocs de la page, drag & drop pour réordonner, bouton pour ajouter un bloc, accès aux paramètres de chaque bloc
- Panneau droit (2/3) : Aperçu en temps réel de la page telle qu'elle sera rendue pour les visiteurs

Édition d'un Bloc

Cliquer sur un bloc ouvre son panneau de configuration dans la partie gauche. Les modifications sont reflétées en temps réel dans l'aperçu. L'édition est non-destructive : une confirmation est demandée avant de supprimer un bloc ou une page.

4.4 Configurateur de Thème

- Couleur principale (primary) et couleur secondaire (secondary)
- Police de titres et police de texte courant (sélection dans une liste de Google Fonts)
- Rayon de bordure global (boutons, cartes, images) : Net / Arrondi / Très arrondi
- Logo (upload) et Favicon (upload)

- Couleur et texte du footer
- Aperçu en temps réel de l'application du thème

5. Gestion des Utilisateurs et des Droits

5.1 Niveaux d'Accès

Rôle	Périmètre	Exemples d'actions
Super Admin	Accès total (agence/dev)	Configurer le thème, activer modules, gérer admins
Admin Client	Gestion du contenu et fonctionnalités	Modifier textes, images, pages, devis, messages
Visiteur	Lecture seule du site public	Consulter les pages, envoyer un formulaire

5.2 Authentification

- Connexion par email + mot de passe (hashé avec bcrypt)
- Session gérée côté serveur avec cookies httpOnly sécurisés
- Déconnexion manuelle + expiration automatique (24h configurable)
- Pas d'inscription publique : les comptes sont créés par le Super Admin
- Réinitialisation de mot de passe par email avec token à durée limitée

5.3 Sécurité

- Protection CSRF sur toutes les mutations
- Rate limiting sur les routes de connexion et les formulaires publics
- Validation et sanitisation des entrées côté serveur (Zod)
- Headers de sécurité : CSP, X-Frame-Options, HSTS (via Nuxt Security module)
- Logs d'accès aux actions critiques (suppression, changement de mot de passe)

6. API Interne

6.1 Principes

L'API est interne au projet Nuxt 4 (server/api/*). Elle suit les conventions REST et retourne du JSON. Toutes les routes protégées vérifient la session en cours via un middleware d'authentification.

6.2 Endpoints Principaux

Endpoint	Méthode	Description
GET /api/pages	GET	Liste toutes les pages (actives/inactives)
POST /api/pages	POST	Créer une nouvelle page
PUT /api/pages/:id	PUT	Modifier une page existante
DELETE /api/pages/:id	DELETE	Supprimer une page
GET /api/blocks/:pageId	GET	Récupérer les blocs d'une page
PUT /api/blocks/:pageId	PUT	Sauvegarder l'ordre et le contenu des blocs
POST /api/contact	POST	Envoyer un formulaire de contact
GET /api/contacts	GET	Lister les messages reçus (admin)
POST /api/quotes	POST	Créer une demande de devis
GET /api/quotes	GET	Lister les devis (admin)
PUT /api/quotes/:id	PUT	Mettre à jour le statut d'un devis
POST /api/upload	POST	Upload de fichier/image
GET /api/settings	GET	Récupérer la configuration globale du site
PUT /api/settings	PUT	Mettre à jour la configuration

7. SEO et Performance

7.1 SEO

- Balises <title> et <meta description> configurables par page
- Open Graph et Twitter Cards (image, titre, description) pour le partage social
- Génération automatique du sitemap XML (sitemap.xml)
- Fichier robots.txt configurable depuis l'admin
- URLs canoniques automatiques
- Données structurées JSON-LD pour les pages clés (optionnel)
- Rendu SSR natif Nuxt 4 pour l'indexation des contenus

7.2 Performance

- Optimisation automatique des images (format WebP, lazy loading, srcset)
- Code splitting par route (natif Nuxt)
- Mise en cache des réponses API avec en-têtes Cache-Control appropriés
- Fonts Google chargées via <link rel=preconnect> et display=swap
- Objectif : Score Lighthouse > 90 sur toutes les métriques

8. Phases de Développement

8.1 Roadmap

Phase	Nom	Contenu	Durée est.
1	Fondations	Init Nuxt 4, auth, BDD, structure fichiers, déploiement CI/CD basique	2 semaines
2	Page Builder	Système de blocs, drag & drop, éditeur inline WYSIWYG, médias	3 semaines
3	Modules métier	Contact, Devis, Galerie, Blog, gestion statuts	2 semaines
4	Thème & SEO	Configurateur couleurs/polices, méta-tags, sitemap, Open Graph	1 semaine
5	Tests & Livraison	Tests E2E, doc utilisateur, formation client, mise en prod	1 semaine

8.2 Phase 1 — Fondations (Détail)

1. Initialisation du projet Nuxt 4 avec TypeScript strict
2. Configuration de TailwindCSS et du système de design (tokens CSS)
3. Mise en place de Drizzle ORM avec schéma initial (users, pages, blocks, settings)
4. Système d'authentification complet (login, session, logout, reset password)
5. Layout d'administration (sidebar, navigation, système de notifications toast)
6. Déploiement initial sur l'environnement cible

8.3 Phase 2 — Page Builder (Détail)

7. Modèle de données : pages ↔ blocs (relation ordonnée)
8. Implémentation du composant éditeur de page avec drag & drop (@vueuse/gesture ou Vue Sortable)

9. Développement des composants de blocs côté public + côté admin
10. Panneau de configuration de bloc (formulaire dynamique selon le type de bloc)
11. Gestionnaire de médias (upload, liste, sélection)
12. Système de prévisualisation en temps réel

9. Contraintes et Exigences

9.1 Exigences Fonctionnelles

- L'administration doit être utilisable sans formation technique par le client
- Toute action destructive (suppression) doit être confirmée
- Les changements non enregistrés doivent déclencher un avertissement
- L'interface doit être responsive (desktop et tablette pour l'admin)
- Le site public doit être parfaitement responsive (mobile first)

9.2 Exigences Techniques

- Compatibilité navigateurs : Chrome, Firefox, Safari, Edge (versions n-1)
- Temps de chargement initial < 2 secondes (LCP)
- Accessibilité : respect des critères WCAG 2.1 niveau AA pour le site public
- Toutes les données doivent être sauvegardées en base de données (pas de fichiers JSON en production)
- Le code doit être documenté et suivre les conventions ESLint + Prettier

9.3 Exigences de Maintenabilité

- Un seul dépôt Git par projet client (fork ou template du projet socle)
- Mises à jour des dépendances facilement applicable (pas de dépendances exotiques)
- Système de migration de BDD Drizzle pour les évolutions de schéma
- Variables d'environnement (.env) pour toute la configuration sensible
- Documentation README complète pour l'installation et le déploiement

9.4 Ce que le système ne fait PAS (hors périmètre v1)

- Éditeur de templates ou de layouts avancés (colonnes personnalisées libres)
- E-commerce / boutique en ligne
- Système de commentaires publics

- Tableau de bord d'analytiques avancées (peut s'intégrer avec Plausible ou Umami)
- Application mobile native

10. Livrables et Critères d'Acceptation

10.1 Livrables Techniques

- Code source complet versionné sur Git (dépôt privé)
- Documentation technique (README, architecture, variables d'env)
- Script de déploiement (Dockerfile ou instructions Vercel/VPS)
- Jeu de données de démonstration (seeder)

10.2 Livrables Fonctionnels

- Site public opérationnel avec pages de démonstration
- Back-office fonctionnel avec compte admin créé
- Documentation utilisateur (guide PDF ou page d'aide intégrée)
- Session de formation client (1h à 2h) incluse

10.3 Critères d'Acceptation

- Le client peut modifier un texte et une image sur n'importe quelle page sans aide
- Le client peut activer/désactiver une page et voir l'effet immédiatement
- Les formulaires de contact et de devis envoient bien les emails
- Le site obtient un score Lighthouse ≥ 90 en Performance, SEO, Accessibilité
- Tous les modules activés fonctionnent sans erreur en production

11. Glossaire

Terme	Définition
Bloc	Unité de contenu visuel composant une page (hero, texte, image, etc.)
Page Builder	Interface permettant de composer des pages par assemblage de blocs sans code
CMS	Content Management System — Système de gestion de contenu

Terme	Définition
SSR	Server-Side Rendering — Rendu HTML côté serveur pour le SEO et les performances
ORM	Object-Relational Mapping — Couche d'abstraction pour interagir avec la base de données
Slug	Version URL d'un titre, ex : 'À propos' → 'a-propos'
Module	Fonctionnalité optionnelle du CMS pouvant être activée ou désactivée
Super Admin	Compte technique de l'agence avec accès total à la configuration
Admin Client	Compte remis au client pour gérer son contenu au quotidien
WYSIWYG	What You See Is What You Get — Éditeur de texte avec aperçu direct du rendu
LCP	Largest Contentful Paint — Métrique de performance mesurant la vitesse d'affichage
WCAG	Web Content Accessibility Guidelines — Normes d'accessibilité web

12. Conclusion

Ce cahier des charges définit un CMS/Page Builder moderne, maintenable et centré sur l'expérience utilisateur du client final. Construit sur Nuxt 4, il tire parti de l'écosystème Vue.js et des meilleures pratiques du développement web contemporain.

La modularité du système (activation/désactivation de fonctionnalités) permet d'adapter la solution à des profils de clients très différents, des artisans locaux aux PME, tout en maintenant un socle de code unique et facile à maintenir pour l'agence.

Ce document est un cadre de travail vivant. Il devra être mis à jour au fur et à mesure des retours client, des arbitrages techniques et des évolutions fonctionnelles identifiées en cours de projet.

Validé par

Client / Commanditaire

Réalisé par

Équipe de développement